Lecture 10
Solutions

Homework 10: Rescue, squash, bisect, shelve

Solutions to Homework 10

```
$ bash codeHints/L10/setup_hw10.sh
$ cd practice/hw10
$ git reflog | grep "Important work"            # 10.1
0b30b44 HEAD@{18}: commit: Important work      (your number differs)
$ git branch rescued 0b30b44                      # use the ID you see
$ git switch messy                                # 10.2
$ git reset --soft main
$ git commit -m "Add feature"
$ git switch history                              # 10.3
$ git bisect start
$ git bisect bad
$ git bisect good $(git rev-list --max-parents=0 history)
$ git bisect run bash test.sh
... is the first bad commit
$ git bisect reset
$ git switch main                                 # 10.4
$ echo "work in progress" >> notes.txt
$ git stash
$ git switch urgent
$ sed -i.bak 's/teh/the/' app.txt && rm app.txt.bak     # or edit by hand
$ git commit -am "Fix urgent bug"
$ git switch main
$ git stash pop
$ git status --short
 M notes.txt
```

For 10.5 (the three IDs are examples, yours differ except the hash of `hello`):

```
$ git cat-file -p main | head -1
tree 8cfff4b...
$ printf 'hello\n' | git hash-object --stdin
ce013625030ba8dba906f756967f9e9ca394464a
$ cat .git/refs/heads/main
1689350...
$ printf 'bad-commit=e238140\ntree=8cfff4b\nhello=ce01362\nref-file=1689350\n' > ../
    hw10_answers.txt
        # (writes all four answer lines at once: bad-commit from 10.3, the rest for
    10.5)
```

10.1. The reset moved `main` back, but the reflog remembers every position of `HEAD`. Your reflog is long because it also contains the history of the setup; `grep` finds the line. `git branch rescued <id>` only adds a label, so `main` stays where it was. (If you used `git reset --hard <id>` on `main`, the rescue works but `main` moved, and the checker complains.)

10.2. `git reset --soft main` moves `messy` back to `main`'s commit but keeps all the changes *staged*, so a single `git commit` makes one commit with exactly the same files as the three wip commits (the checker compares the file contents).

10.3. `bisect run` tests about $\log_2 12 \approx 4$ commits. The first bad commit is `Step 7`. Do not forget

`git bisect reset`: otherwise the repository stays in bisect mode and the checker reports it.

10.4. A stash lets you switch branches with a clean working directory. After the pop, your uncommitted line is back in `notes.txt`, and the commit `Fix urgent bug` lives on `urgent`, not on `main`. If `git stash pop` reports a conflict, you changed the same lines on both sides; resolve it as in Lecture 5.

10.5. `git cat-file -p main` prints a commit: its first line gives the ID of its tree. The hash of `hello` is the same on every computer (`ce01362...`). The file `.git/refs/heads/main` holds the full ID of `main`'s commit, so its first 7 characters equal `git rev-parse --short=7 main`. (If the file does not exist, Git has packed the reference into `.git/packed-refs`; use `git rev-parse main` instead.)

Common mistakes.

- Moving `main` to the lost commit in 10.1 instead of creating a branch.
- Forgetting `git bisect reset`; or running bisect while having uncommitted changes (stash or commit first).
- Committing the `work in progress` line instead of stashing it in 10.4: then `main` has a new commit and the check "only `M notes.txt`" fails.
- Writing fewer than 7 characters of an ID: write at least 7 (the full 40 are accepted too).